VC Policy Engine — Design & Architecture Document

Nov 8, 2026 · @Suresh Unnikrishnan

Overview

VC Policy Engine is a TypeScript platform that turns a verifiable presentation into a governed PASS, FAIL or REVIEW decision. It runs as an embeddable SDK, a standalone service, and a multi-tenant relying-party intermediary service. This document is the v1.0 design baseline. It replaces Blueprint v0.1 wherever the two differ.

Status: Final baseline v1.0, pending sign-off · **Supersedes:** Architecture & Implementation Blueprint v0.1

Goals

1. Verify SD-JWT VC and ISO mdoc presentations over OpenID4VP 1.0 with HAIP 1.0, including through the W3C Digital Credentials API.
2. Establish trust from EU trusted lists and X.509, with issuer authority scoped to credential types and claims.
3. Evaluate versioned business policies deterministically and return explained decisions with minimal, privacy-safe evidence.
4. Run multi-tenant as a relying-party intermediary, with per-tenant registration certificates and hardware-held keys.
5. Add W3C VC 2.0 and Inji credentials (India) and B2B trust frameworks (vLEI, EU Business Wallet, UNTP, TRQP, OpenID Federation) through adapters, without changing the core.

Non-goals (v1.x)

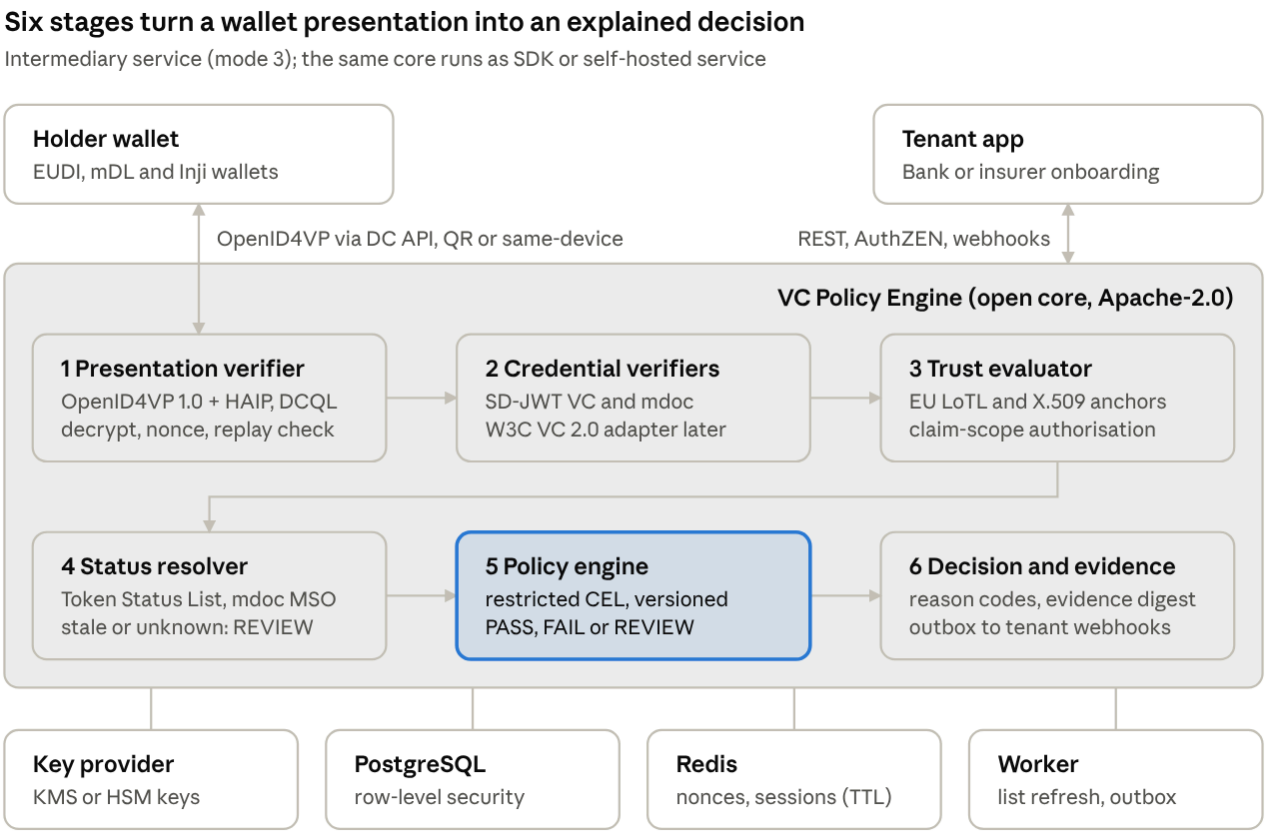
- Not an identity provider, custodial wallet, payment processor or full credential issuer (a demo issuer only).
- Not a replacement for AML screening, sanctions checks or document KYC; decisions feed those processes.
- No execution of untrusted policy code; no self-hosted model inference; no blockchain dependency.
- No claim to establish the real-world truth of a credential beyond its issuer's authority.

Decisions that shape the design

Decision	Design consequence
EU relying parties first, then India DPI, then global B2B	P0 formats are SD-JWT VC and mdoc; trust is X.509/LoTL first; W3C VC and DID trust are adapters
Act as, or partner with, a relying-party intermediary	Multi-tenancy, KMS/HSM keys and per-tenant registration certificates are in v1.0; no transaction content is stored
Open core, then foundation	All verification, trust and policy logic is Apache-2.0; commercial code is operational only and lives in separate repositories
A bank or insurer as design partner	Fail-closed semantics, a REVIEW queue, audit replay, DORA-ready operations
Reuse OpenWallet Foundation libraries	Protocol and format code is wrapped behind ports, not rewritten

Architecture at a glance

The engine is a fixed pipeline of six stages behind two interfaces: a wallet-facing protocol side and a tenant-facing API side. The policy engine (highlighted) is the product's core value. Every other stage exists to give it trustworthy, typed input.



component architecture · 6 stages, 4 infrastructure adapters

Stages 1–4 can only reject or downgrade to REVIEW. Only stage 5 can return PASS. Infrastructure plugs in through ports, so the same core runs in all three deployment modes.

Deployment modes and intermediary topology

The decision logic is identical in every mode. Modes differ only in which adapters are plugged into the ports.

Mode	Who runs it	Storage	Keys	Typical user
1. Embedded SDK	The relying party, inside its own app	In-memory or the app's own repositories	The app's key provider	Developers, prototypes, edge services
2. Standalone service	The relying party, self-hosted	PostgreSQL	KMS or HSM via PKCS#11	A bank that keeps verification in-house

Mode	Who runs it	Storage	Keys	Typical user
3. Intermediary service (multi-tenant)	Us, or a licensed partner, on behalf of many relying parties	PostgreSQL, partitioned per tenant	Cloud KMS/HSM; one access certificate per intermediary instance, registration certificates per tenant	Banks and insurers buying the managed service

Intermediary rules (mode 3)

- **Who faces the wallet.** The intermediary authenticates to wallets with its own access certificate, and selects the registration certificate of the tenant it acts for on each request.
- **What is kept.** An intermediary may not store data about the content of a transaction (Didit summary of Art. 5b(10)). The service returns verified claims to the tenant and keeps only decision metadata: outcome, reason codes, policy version, trust snapshot and hashes.
- **Tenant isolation.** Every row carries `tenant_id`. PostgreSQL row-level security enforces isolation, and API tokens are bound to one tenant. Policies, trust configuration, webhooks and keys are separate per tenant.
- **Region.** EU tenants run in an EU region. India tenants run in India, so DPDP Act data rules can be met.
- **Tenant-facing data.** Claims are returned once, over the tenant's authenticated channel (an API response or a signed webhook payload), and are never written to disk.

Runtime topology (mode 3)

- `api` (Fastify): tenant API, AuthZEN endpoint, wallet-facing OpenID4VP endpoints. Stateless; scaled horizontally.
- `worker`: status-list and trusted-list refresh, webhook delivery from the outbox, retention jobs.
- PostgreSQL (primary + replica), Redis for nonces and session state with TTLs, and a KMS/HSM for signing and decryption keys.
- Admin console and Policy Studio (React), using the same APIs; no verification logic in the browser.

Packages and components

The repository is a pnpm workspace with hexagonal boundaries: core packages depend only on ports, never on Fastify, PostgreSQL, React or a specific crypto library. Packages marked *later* arrive in P4–P5.

Package	Responsibility	Built on	Phase
`core`	Domain types, typed errors, ports (crypto, keys, trust, status, storage, clock), orchestration	—	P0
`crypto`	JOSE/COSE primitives, X.509 path validation, algorithm allow-list	`jose`, `@peculiar/x509`, WebCrypto	P1
`format-sd-jwt-vc`	Parse and verify SD-JWT VC, disclosures, KB-JWT, `vct` type metadata	OWF `sd-jwt-js`	P1
`format-mdoc`	Verify ISO 18013-5 mdoc: MSO, digests, issuer auth, device auth	Chosen mdoc library (ADR-004)	P1
`format-w3c-vc`	W3C VC 2.0: JOSE/COSE securing and one Data Integrity suite	Digital Bazaar libraries	P4 (*later*)
`openid4vp`	Authorization requests (JAR, DCQL, `transaction_data`), response decryption, correlation, replay checks	OWF `openid4vc-ts`, `dcql-ts`	P1
`dc-api`	W3C Digital Credentials API requests (OpenID4VP and ISO 18013-7 Annex C)	`openid4vp`, `format-mdoc`	P2
`trust`	Trust anchors, claim-scope authorisation, trust-config snapshots	Adapters below	P1
`trust-eu-lotl`	EU List of Trusted Lists / ETSI TS 119 612 loader and X.509 anchor store	`crypto`	P1
`trust-trqp`, `trust-oidfed`, `trust-did`	ToIP TRQP v2.0, OpenID Federation, DID resolution	OWF `openid-federation-ts`	P5 (*later*)

Package	Responsibility	Built on	Phase
`status`	Token Status List (JWT/CWT), mdoc MSO revocation, Bitstring Status List; freshness and caching	`crypto`	P1 (Bitstring P4)
`policy`	Policy JSON schema, compiler, type checker, evaluator, simulator	—	P1
`policy-cel`	Restricted CEL profile	`cel-js` (ADR-006)	P1
`evidence`	Decision records, reason codes, evidence digests, retention	—	P1
`tenancy`	Tenants, registration certificates, per-tenant keys and config	—	P2
`storage-memory`, `storage-postgres`	Repository adapters, migrations, row-level security	`pg`, `kysely`	P0 / P2
`keys-kms`	KMS/HSM key provider (AWS KMS, GCP KMS, PKCS#11)	Cloud SDKs	P2
`sdk`	Public facade `createVcPolicyEngine(...)`	All of the above	P1
`testing`	Fake issuers, fixtures, test vectors, conformance harness	—	P0 onward
`apps/api`	Fastify service: tenant REST, AuthZEN, wallet endpoints, OpenAPI	`sdk`	P2
`apps/worker`	Refresh jobs, outbox delivery, retention	`sdk`	P2
`apps/console`	Policy Studio, trust admin, decision explorer, REVIEW queue (React)	`api`	P2
`apps/demo-wallet`, `apps/demo-issuer`	Reference wallet and issuer for local demos	`sdk`	P1

Library choices marked with an ADR are confirmed in P0 by a short spike.

Decision flow

A decision moves through four gates in a fixed order: presentation, credential, trust, policy. A failure in any of the first three can never produce PASS.

1. **Start session.** The tenant app calls `POST /v1/sessions` with a policy id and an idempotency key. The engine pins the published policy version and derives the DCQL query from its required evidence.
2. **Build the request.** The engine creates a nonce, `state` and expiry. It signs the OpenID4VP request object (JAR) with the intermediary's access-certificate key (`x509_hash` client id). It attaches the tenant's registration certificate and an encryption key for the response.
3. **Deliver to the wallet.** Through the Digital Credentials API where the browser supports it; otherwise a QR code or same-device redirect with `request_uri`.
4. **Receive the response.** The wallet posts `direct_post.jwt`. The engine decrypts it, checks `state`, nonce, audience and expiry, and rejects any replay (nonce is single-use in Redis).
5. **Verify credentials.** Each credential goes to its format verifier (SD-JWT VC or mdoc). The verifier checks the issuer signature and X.509 chain, disclosures or digests, validity period, and holder/device binding against the request.
6. **Evaluate trust.** The trust evaluator checks that the issuer is anchored in the tenant's trust configuration (for example an EU trusted list) and is authorised for this credential type and these claim paths.
7. **Check status.** The status resolver fetches or reuses a cached status list within the policy's freshness limit. Revoked gives FAIL; unknown or unavailable gives REVIEW.
8. **Normalise claims.** Verified claims are mapped into typed bindings (e.g. `person.age_over_18`, `address.country`). Unverified input never reaches this step.
9. **Evaluate policy.** Restricted CEL rules run against the bindings and trusted business context, with a fixed clock. Each rule yields PASS, FAIL or REVIEW with a reason code.
10. **Record and notify.** The engine writes the decision record and evidence digest in one transaction with an outbox event. It returns the decision and verified claims to the tenant by API or signed webhook. It then deletes session data.

Failure paths. Steps 4–5 fail with typed verification errors and no decision. Steps 6–7 can produce FAIL (revoked, unauthorised issuer) or REVIEW (status unavailable, missing evidence). The REVIEW queue in the console shows the reason codes but not the claims.

Trust architecture

Trust has two layers. Anchors answer "is this key really this issuer's?" Claim-scope rules answer "is this issuer authorised for this claim?" A valid signature from an anchored issuer still fails if the claim is out of scope.

Inbound trust (issuers)

Anchor source	Used for	Adapter	Phase
EU List of Trusted Lists / national trusted lists (ETSI TS 119 612)	PID and (Q)EAA issuers in the EUDI ecosystem	`trust-eu-lotl`	P1
Pinned X.509 roots per tenant	Private ecosystems, sandboxes, mDL roots (e.g. an AAMVA-style trust list)	`trust` core	P1
SD-JWT VC issuer metadata (JWKS over HTTPS)	Issuers that publish keys, not certificates	`trust` core, allow-listed hosts only	P1
ToIP TRQP v2.0 registries	Ecosystem authorisation queries ("is X authorised for Y under Z")	`trust-trqp`	P5
OpenID Federation trust chains	Federated ecosystems	`trust-oidfed`	P5
DID documents (`did:web` , others by ADR)	W3C VC ecosystems, India DPI, UNTP	`trust-did`	P4–P5

Claim scope. Each `IssuerAuthorization` row grants an issuer (or a trusted-list service type) a set of credential types (`vct` or mdoc `doctype`) and claim paths. Example: a university is trusted for degree claims but not for insurance cover. Trust configuration is versioned, and every decision records the version it used.

Outbound trust (the verifier itself)

- **Access certificate.** Issued to the relying party or intermediary instance by an authorised CA. Its private key stays in KMS/HSM and signs every request object (HAIP `x509_hash` client id; `x509_san_dns` accepted for older wallets as an option).
- **Registration certificate.** One or more per tenant, stating the intended use and the registered attributes. The engine checks that a policy's required claims fit within the selected registration before it sends the request.

- **Response encryption keys.** Ephemeral or per-tenant ECDH-ES keys from KMS, rotated on a schedule.

Refresh and failure

The worker refreshes trusted lists and status lists on a schedule and keeps the last valid copy with its signature and fetch time. If a list is stale beyond policy limits, trust or status comes back `unavailable`, and the decision becomes REVIEW, never PASS. All outbound fetches go through an allow-list with size, time and redirect limits (SSRF control).

Formats, protocols and conformance matrix

v1.0 targets the HAIP 1.0 verifier profile for both SD-JWT VC and mdoc. Everything else is either a later adapter or explicitly not supported. The matrix below is published in the repository and updated on every release.

Area	v1.0 (P1–P2)	Later	Not supported
Presentation protocol	OpenID4VP 1.0 with HAIP 1.0; DCQL; JAR; `direct_post.jwt`; `transaction_data`	OpenID4VP over BLE (proximity)	DIF Presentation Exchange; SIOPv2
Delivery channel	W3C Digital Credentials API (OpenID4VP, ISO 18013-7 Annex C); QR / same-device `request_uri`	—	Custom deep-link schemes
Credential formats	SD-JWT VC (`dc+sd-jwt`, `vc+sd-jwt` accepted during transition); ISO mdoc (18013-5)	W3C VC 2.0 (JOSE/COSE, one Data Integrity suite) in P4; CWT/QR (Inji) in P4	AnonCreds; BBS and ZKP (research track)
Holder binding	KB-JWT (SD-JWT VC); mdoc device authentication	W3C VP proofs (P4)	Bearer credentials for any policy that asks for binding
Algorithms	ES256 required; ES384/ES512 and EdDSA allow-listed per tenant	—	`none`, RSA < 2048, HS*

Area	v1.0 (P1–P2)	Later	Not supported
Verifier identity	`x509_hash` (HAIP); `x509_san_dns` option	`openid_federation`, `decentralized_identifier`	`redirect_uri` client id in production
Status	IETF Token Status List (JWT/CWT); mdoc MSO revocation	W3C Bitstring Status List (P4)	OCSP for credentials
Trust anchors	EU LoTL / TS 119 612; pinned X.509 roots; JWKS issuer metadata	TRQP v2.0, OpenID Federation, DIDs	Unpinned web PKI alone
Decision API	REST `/v1`; OpenID AuthZEN Authorization API 1.0	—	—

Conformance targets. OIDF OpenID4VP 1.0 + HAIP verifier self-certification at G2 (OIDF). The OIDF suite runs in CI on every merge to `main`. Interop runs against the EU reference wallet, Credo-based wallets and at least three national wallets in P3.

Policy model

A policy is a versioned JSON document. It declares the evidence it needs and lists rules written in a restricted CEL profile. The engine compiles and type-checks it before publication and evaluates it deterministically against typed bindings.

Semantics

- **PASS:** every mandatory rule holds on authenticated, trusted, fresh evidence.
- **FAIL:** authenticated evidence shows that a mandatory rule does not hold (e.g. revoked, under age, sanctioned country flag from trusted context).
- **REVIEW:** evidence is missing, stale, unavailable or conflicting, or a rule is marked for human judgement.
- **Aggregation:** any FAIL gives FAIL; otherwise any REVIEW gives REVIEW; otherwise PASS. Verification errors return no decision.

Restricted CEL profile

- Inputs are only the typed bindings from verified claims and a typed, tenant-supplied `context` (e.g. product, risk tier).

- No network calls, no host functions, no regex beyond a safe subset, no user-defined functions.
- Money is integer minor units with an explicit currency, and dates are ISO dates. The evaluator receives `now` from the request clock.
- Expressions have a cost limit, checked at compile time.

Lifecycle and governance

`DRAFT → VALIDATED → APPROVED → PUBLISHED → DEPRECATED → RETIRED`. Published versions are immutable. Publishing needs two different people (author and approver) and passing attached test cases. Sessions pin the version they started with. Rollback means publishing an earlier definition as a new version. Simulation runs a draft against synthetic or consented historical decision inputs and shows how outcomes would change.

Example: retail bank onboarding (illustrative)

```
{
  "id": "retail-onboarding-eu",
  "version": "1.0.0",
  "registration": "rc-retail-onboarding",
  "requiredEvidence": [
    { "id": "pid", "type": "eu.europa.ec.eudi.pid.1",
      "formats": ["dc+sd-jwt", "mso_mdoc"],
      "claims": ["family_name", "given_name", "birth_date", "nationality",
"resident_country"],
      "required": true, "maxStatusAgeSeconds": 86400 }
  ],
  "rules": [
    { "id": "adult", "expression": "age(pid.birth_date, now) >= 18",
      "onFalse": "FAIL", "reasonCode": "UNDER_AGE" },
    { "id": "eea-resident", "expression": "pid.resident_country in
context.allowedCountries",
      "onFalse": "REVIEW", "reasonCode": "RESIDENCE_OUTSIDE_SCOPE" },
    { "id": "high-risk-product", "expression": "context.riskTier != 'HIGH'",
      "onFalse": "REVIEW", "reasonCode": "ENHANCED_DUE_DILIGENCE" }
  ]
}
```

Claim names and the PID type follow the EUDI PID rulebook in spirit. The exact names are confirmed with the design partner in P0.

Interfaces

There are four interface families, each with its own authentication: tenant REST, AuthZEN decisions, wallet-facing protocol endpoints, and outbound webhooks. The SDK exposes the same operations as functions.

Tenant REST API (`/v1` , OAuth 2.0 client credentials, tenant-bound tokens)

Endpoint	Purpose
`POST /sessions`	Start a presentation session for a policy (idempotency key required)
`GET /sessions/{id}`	Session state: `AWAITING_PRESENTATION` , `VERIFYING` , `DECIDED` , `EXPIRED` , `REJECTED`
`GET /sessions/{id}/decision`	The decision; verified claims are available once, then purged
`POST /evaluations`	Evaluate already-verified evidence (SDK parity, used by integrators)
`GET /decisions/{id}`	Decision record with reason codes, policy version and evidence digest (no claims)
`POST /policies` , `POST /policies/{id}/validate` , `/simulate` , `/approve` , `/publish` ; `GET /policies/{id}/versions`	Policy lifecycle
`GET/POST /trust/issuers` , `/trust/authorizations` , `/trust/anchors`	Trust configuration (versioned)
`GET/POST /registrations`	Tenant registration certificates and their registered attributes
`GET /review-queue` , `POST /review-queue/{id}/resolve`	REVIEW handling by named reviewers, with notes

OpenID AuthZEN decision endpoint

`POST /access/v1/evaluation` takes `subject` (the session or evidence reference), `action` (e.g. `onboard`), `resource` (product) and `context` . It returns `decision: true/false` plus a `context.outcome` of PASS, FAIL or REVIEW and reason codes. REVIEW maps to `false` with a review reason, so existing policy enforcement points fail closed.

Wallet-facing endpoints (protocol-defined, not generic JSON)

- `GET /wallet/requests/{id}`: signed request object (JAR).
- `POST /wallet/responses/{id}`: `direct_post.jwt` response.
- Digital Credentials API: the request is built server-side, and the page passes it to `navigator.credentials.get` and returns the result to the response endpoint.

Webhooks and events

Events are written to an outbox and delivered with HMAC signatures, retries and idempotency keys: `presentation.requested`, `presentation.received`, `presentation.rejected`, `decision.finalized`, `decision.review_required`, `credential.status_unavailable`, `trust.list_stale`, `policy.published`. Payloads carry ids and outcomes. Claims are sent only in `decision.finalized`, and only where the tenant has chosen webhook delivery.

Data model, persistence and privacy

The database holds configuration and decision metadata, never credentials or raw presentations. Session data lives only in Redis with a TTL and is deleted when the decision is recorded.

Table	Key columns	Retention
`tenants`	id, region, status, settings	Life of contract
`registrations`	tenant_id, cert_ref, intended_use, registered_attributes[]	Life of certificate
`policies`, `policy_versions`	tenant_id, policy_id, version, status, definition_json, author, approver, published_at	Indefinite (immutable once published)
`policy_test_cases`	policy_version_id, input_fixture, expected_outcome	With policy
`trust_anchors`, `issuer_authorizations`, `trust_config_versions`	tenant_id, anchor/issuer, credential_types[], claim_paths[], valid_from/until	Versioned; indefinite

Table	Key columns	Retention
`decision_records`	id, tenant_id, session_ref, outcome, reasons[], policy_version_id, trust_config_version_id, evidence_digest, formats[], issuer_refs[], status_results[], created_at	Tenant-configured; default 5 years for AML evidence
`review_items`	decision_id, assignee, resolution, note, resolved_at	With decision
`status_cache`, `trusted_list_cache`	source_url, payload, signature, fetched_at, expires_at	Until replaced
`outbox_events`, `audit_events`	tenant_id, type, payload_ref, actor, at	Audit: append-only; tenant-configured

Evidence digest. A SHA-256 hash over a canonical record: request hash, response hash, credential hashes, disclosed claim paths, trust and status results, and policy version. The tenant, which held the claims, can later prove what was decided without our storing the content.

Privacy controls:

- Data minimisation: requests ask only for the policy's claims, and only within the tenant's registration. The decision record stores claim *paths*, not values.
- No claims in logs or metric labels; identifiers pseudonymised; field-level encryption for review notes.
- Row-level security per tenant; separate EU and India regions; deletion jobs for retention and data-subject requests.
- The default 5-year retention follows common AML record-keeping practice. Each tenant confirms its own period.

Security architecture

The system fails closed at every trust boundary. No cryptographic, trust or status failure can turn into PASS, and no browser-side result can drive an approval.

Trust boundaries

1. Tenant app → engine (OAuth client credentials, mTLS optional)
2. Engine → wallet (signed requests, encrypted responses)

3. Credential bytes → format verifiers (untrusted input)
4. Engine → trusted lists, issuer metadata, status lists (outbound fetches)
5. Verified claims → policy evaluator (typed bindings only)
6. Engine → database, KMS/HSM, webhooks

Threats and controls

Threat	Control
Forged or altered credential	Signature and X.509 chain validation; disclosure digest checks; algorithm allow-list
Untrusted or out-of-scope issuer	Anchor check plus claim-scope authorisation; never PASS on signature alone
Replay or response injection	Single-use nonce, `state` binding, audience check, short expiry, encrypted responses
Holder impersonation	KB-JWT and mdoc device authentication, required by policy
Verifier impersonation (towards wallets)	Signed requests with access certificate; keys never leave KMS/HSM
Revoked credential accepted	Status checks with freshness limits; unavailable gives REVIEW
SSRF through metadata or list URLs	Allow-listed hosts, HTTPS only, size and time limits, no redirects to private ranges
Malicious CBOR/JSON (parser attacks)	Size and depth limits; fuzzing in CI; no JSON-LD document loading in v1.0
Policy injection or runaway expressions	Typed AST, restricted CEL, compile-time cost limits, no host functions
Cross-tenant data access	Tenant-bound tokens; row-level security; per-tenant keys and webhooks; tests in CI
Insider policy change	Two-person publish; immutable versions; audit events
Claim leakage	No claims at rest; no claims in logs or metrics; one-time claim retrieval

Key management

Key	Where	Rotation
Access-certificate signing key	HSM or cloud KMS (non-exportable)	With certificate renewal
Response-decryption keys	KMS; ephemeral per session where the wallet supports it	Daily or per session
Webhook HMAC secrets	Secret manager, per tenant	On demand and every 90 days
Database encryption	Managed at-rest encryption plus field-level keys in KMS	Yearly

Assurance

Threat-model review every phase. Dependency and container scanning on every build. Parser fuzzing. An external pen test before G2 and yearly afterwards. SBOM and signed release provenance. A DORA third-party due-diligence pack (incident reporting, exit plan, sub-processor list) for financial-sector tenants, and an ISO 27001 path for the managed service.

Operations and observability

Operations are built for an intermediary that must report incidents to regulated customers. Every decision is traceable by correlation id without exposing claims.

Metrics (no claims or holder ids as labels):

- Decision latency (end to end) and policy evaluation latency
- Verification latency by format; trust-resolution latency
- PASS / FAIL / REVIEW rates per policy version
- Verification errors by code; rejected replays
- Status-cache and trusted-list freshness
- Session completion and expiry rates by channel (DC API, QR, same-device)
- Webhook delivery lag

Service targets (to confirm with the design partner):

Measure	Target
Policy evaluation, p95 (excluding I/O)	< 20 ms
Verification + decision after wallet response, p95 (warm caches)	< 500 ms
Service availability (managed)	99.9% monthly
Trusted-list and status-list staleness	Within policy limits; alert at 80% of limit

Runbooks: trusted-list fetch failure, status-provider outage (REVIEW surge), key compromise and certificate revocation, wallet spec change, tenant offboarding and data export.

Release process: semantic versioning, changesets, the OIDF conformance suite and the golden end-to-end test on every release, staged rollout for the managed service, signed artefacts.

Architecture decision records

These records fix the main choices. Each one is a file in `docs/adr/`. Changing a decision means writing a new ADR that supersedes the old one.

ADR	Decision	Status
ADR-001	TypeScript (strict), Node.js 22+, pnpm monorepo, hexagonal ports and adapters	Accepted
ADR-002	Wrap OWF `openid4vc-ts`, `dcql-ts`, `sd-jwt-js` rather than re-implement	Accepted
ADR-003	SD-JWT VC and mdoc both P0; W3C VC 2.0 as P4 adapter	Accepted
ADR-004	mdoc verification library (candidates to compare in a P0 spike)	Open, P0
ADR-005	Trust: EU LoTL / X.509 first; TRQP, OpenID Federation, DID as adapters	Accepted
ADR-006	Policy: JSON + restricted CEL; library choice in P0 spike	Accepted (library open)

ADR	Decision	Status
ADR-007	Three-outcome decisions; fail closed; REVIEW on unavailable evidence	Accepted
ADR-008	Multi-tenant intermediary in v1.0; no transaction content at rest	Accepted
ADR-009	Keys in KMS/HSM via a key-provider port; PKCS#11 for on-prem	Accepted
ADR-010	IETF Token Status List first; Bitstring Status List in P4	Accepted
ADR-011	OpenID AuthZEN as the standard decision API alongside REST	Accepted
ADR-012	Apache-2.0 with DCO; commercial code in separate repositories	Accepted

References

Related documents: Business Opportunity Report, Implementation Plan, Opportunity, Industry & Interoperability Report. Input: Architecture & Implementation Blueprint v0.1.

- HAIP 1.0 and OIDF final specifications · OIDF conformance tests · OpenID4VP + DC API security analysis
- SD-JWT VC draft-18 · IETF Token Status List · W3C VC 2.0 family
- Chrome Digital Credentials API · ToIP TRQP v2.0
- German EUDI RP onboarding · BMI status management · EU reference verifier · Didit — intermediary rules
- OWF TypeScript projects